

Assignment 4

SSH Key-Based Authentication & Secure File Transfer

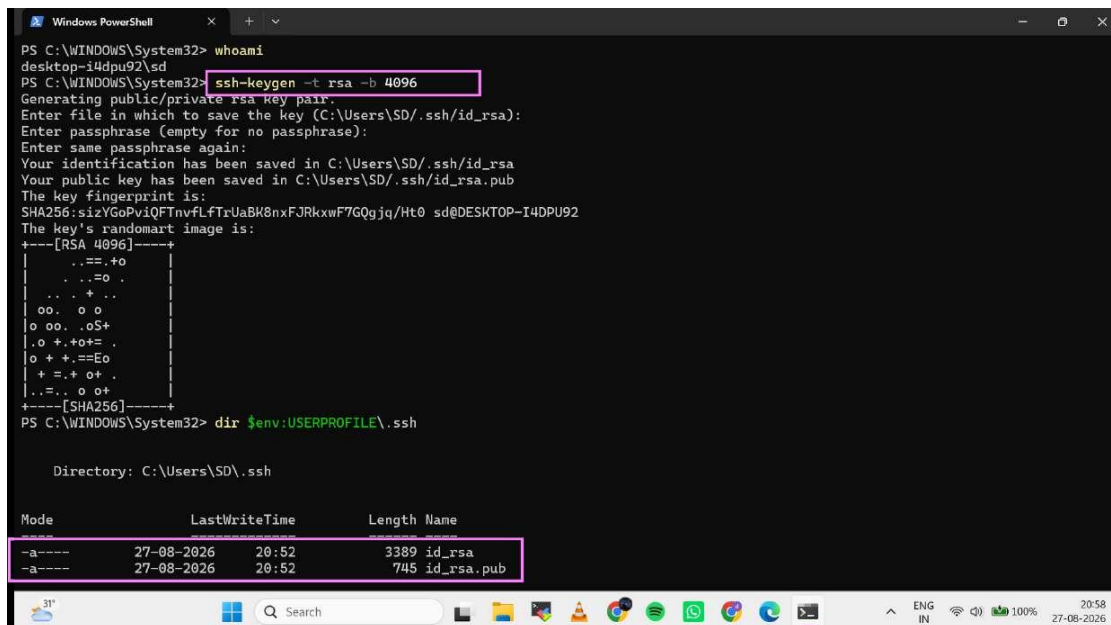
Local Machine (Windows / PowerShell) → Remote Server (AWS EC2 – Amazon Linux 2023)

1. Scenario

The goal of this assignment is to configure secure, passwordless SSH access from a local Windows machine to a remote Linux (AWS EC2) server, and to securely transfer a sensitive configuration file to and from the server using SCP, without using password-based authentication at any point.

2. Step 1 – Generate the SSH Key Pair (Local Machine)

- **Command used:** `ssh-keygen -t rsa -b 4096` (generates a 4096-bit RSA key pair)
- A passphrase was entered when prompted, so the private key is itself password-protected.
- The key pair was saved to the default location: `C:\Users\SD\.ssh\`
- Verified with: `dir $env:USERPROFILE\.ssh` — confirms `id_rsa` (private key) and `id_rsa.pub` (public key) were created.



```
PS C:\WINDOWS\System32> whoami
desktop-i4dpu92\sd
PS C:\WINDOWS\System32> ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (C:\Users\SD\.ssh\id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\SD\.ssh\id_rsa
Your public key has been saved in C:\Users\SD\.ssh\id_rsa.pub
The key fingerprint is:
SHA256:sizYGpviQFTnvFLTrUaBK8nxFJRkxwF7GQgjq/Ht0 sd@DESKTOP-I4DPU92
The key's randomart image is:
+--[RSA 4096]-----+
|  ..==+o |
|  ..=o  |
|  .+.  |
| oo. o o |
| o oo. .oS+ |
| .o +.+++= |
| o +.==Eo |
| +.=+ o+ |
| .. . o o+ |
+--[SHA256]-----+
PS C:\WINDOWS\System32> dir $env:USERPROFILE\.ssh

Directory: C:\Users\SD\.ssh

Mode                LastWriteTime         Length Name
----                -
-a-----         27-08-2026    20:52         3389 id_rsa
-a-----         27-08-2026    20:52          745 id_rsa.pub
```

Fig. 1 — `ssh-keygen` generating a 4096-bit RSA key pair and `dir` output showing `id_rsa` / `id_rsa.pub` in `~/.ssh/`

Purpose of the Public and Private Keys

- **Private key (`id_rsa`):** Stays only on the local machine, is protected by the passphrase, and is used to cryptographically prove identity during login. It must never be shared.
- **Public key (`id_rsa.pub`):** Safe to share freely. It is copied to the remote server and placed in the account's `authorized_keys` file, where it acts as a “lock” that only the matching private key can “open.”
- **Result:** During authentication, the server challenges the client to prove possession of the private key without the key ever leaving the local machine — this is what enables secure, passwordless login.

3. Step 2 – Copy the Public Key to the Remote Server

On the remote EC2 server (user ec2-user), the public key contents were placed into the authorized_keys file and locked down with the correct permissions:

```
vi ~/.ssh/authorized_keys # pasted the contents of id_rsa.pub
chmod 700 ~/.ssh          # only the owner can access the .ssh folder
chmod 600 ~/.ssh/authorized_keys # only the owner can read/write this file
```

SSH refuses to use authorized_keys if the folder/file permissions are too open, so this step is mandatory, not optional.

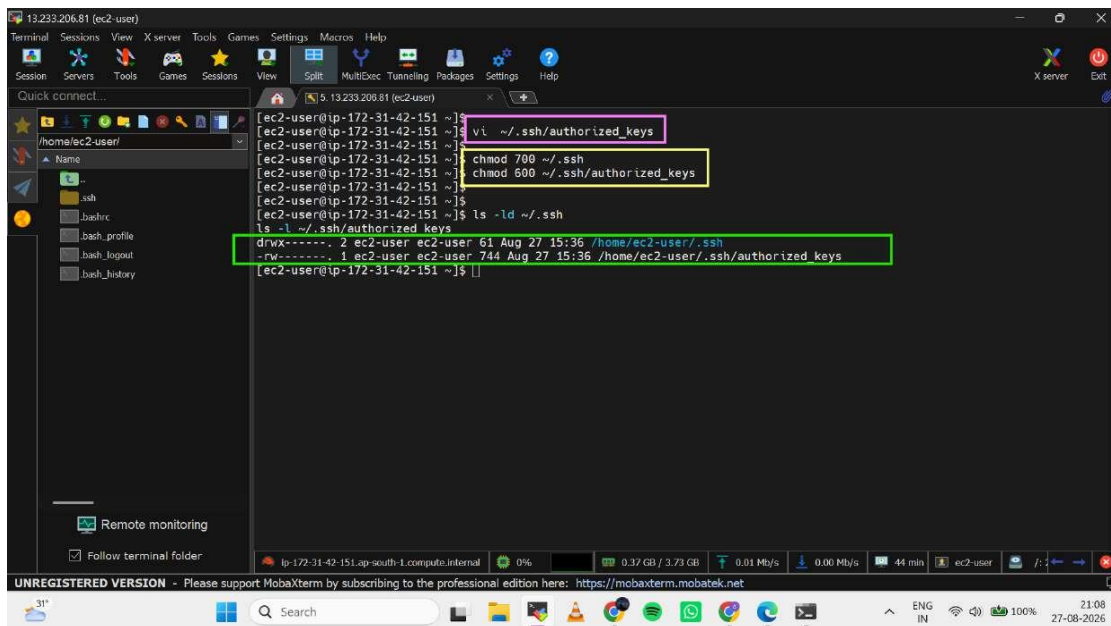


Fig. 2 — Creating ~/.ssh/authorized_keys, setting chmod 700 / 600, and ls -ld confirming the permissions

Verifying the Key Was Copied Correctly

On the remote server, the file was inspected to confirm the key content and file permissions:

```
cat ~/.ssh/authorized_keys # confirms the public key text is present
ls -l ~/.ssh/              # confirms authorized_keys=600, id_rsa.pub=644
```


5. Step 4 – Secure File Transfer with SCP

5.1 Create the Sensitive File (Local Machine)

A dummy sensitive configuration file was created locally using Notepad:

```
notepad config.yaml
```

(see Fig. 3 above — the notepad config.yaml command opens the file for editing before it is transferred)

5.2 Transfer config.yaml to the Remote Server

- **Command used:** `scp -i "$env:USERPROFILE\.ssh\id_rsa" config.yaml ec2-user@13.233.206.81:~/`

This securely copies config.yaml from the local machine into the ec2-user home directory over the encrypted SSH channel, authenticated using the same key pair — no password prompt appears.

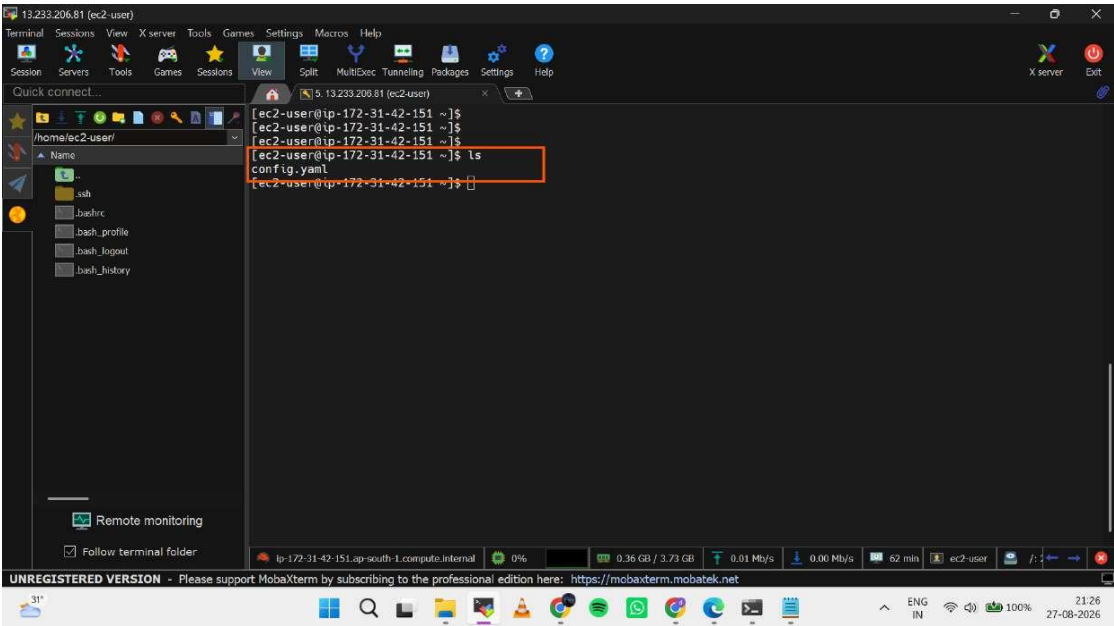


Fig. 5 — ls on the remote server confirming config.yaml has arrived in the home directory

5.3 Retrieve a Log File from the Remote Server

- **Command used:** `scp -i "$env:USERPROFILE\.ssh\id_rsa" ec2-user@13.233.206.81:/var/log/syslog ~/`

This pulls the remote log file back down to the local machine's home directory, again fully encrypted and authenticated with the SSH key — completing a secure, bidirectional file transfer workflow without ever typing a password.

6. Verification Summary

Verification Item	Status / Evidence
Key generation command & output	ssh-keygen -t rsa -b 4096 — Fig. 1
ls -l ~/.ssh/ (local machine)	id_rsa (private) and id_rsa.pub (public) present — Fig. 1
cat ~/.ssh/authorized_keys (remote)	Public key content confirmed present — Fig. 3
Permissions: .ssh = 700, authorized_keys = 600	Confirmed via chmod + ls -ld — Fig. 2
Passwordless SSH login demonstrated	Direct shell access with no password prompt — Fig. 4
ls ~/config.yaml (remote, after transfer)	config.yaml present in home directory — Fig. 5
scp retrieval of /var/log/syslog	Command executed against the remote host using the same

Verification Item	Status / Evidence
	key-based authentication

7. Key Concepts — SSH Key Pair Components

- **Asymmetric key pair:** RSA generates two mathematically linked keys — what one encrypts, only the other can decrypt.
- **Private key (id_rsa):** Kept secret on the client, optionally passphrase-protected, used to sign the authentication challenge.
- **Public key (id_rsa.pub):** Distributed to any server the user needs to access; stored in that account's `~/.ssh/authorized_keys`.
- **authorized_keys:** A per-user allow-list on the server — each line is a public key permitted to log in as that user.
- **Passphrase:** An extra local layer of protection on the private key itself, independent of the server-side authentication.
- **Strict permissions (700 / 600):** SSH will silently reject a key-based login if the `.ssh` directory or `authorized_keys` file is writable by anyone other than the owner.
- **SCP over SSH:** File transfers are tunnelled through the same encrypted, key-authenticated SSH session, so no separate credentials or protocols are needed.